PEMROGRAMAN SISI SERVER

Minggu 3 — Docker Containerization

Suprayogi,M.Kom

A11.64707 | 3 SKS | Semester 6

Tujuan Pembelajaran Minggu 3

- Menjelaskan image, container, Dockerfile, registry, volume, network, dan Compose.
- Menjalankan container menggunakan Docker CLI.
- Membangun custom image dari Dockerfile.
- Menggunakan environment variable tanpa menanam credential di source code.
- Menghubungkan application container dan PostgreSQL melalui Docker Network.
- Memahami persistent data dan konsep multi-stage build.
- Menghubungkan praktik Docker dengan fondasi project Simple LMS untuk Minggu 4.

Apa Itu Containerization?

- Teknik mengemas aplikasi beserta dependency dan runtime ke dalam lingkungan terisolasi yang disebut container.
- Tujuan utama: konsistensi, isolasi, portabilitas, distribusi, dan reproducibility.
- Application + Dependencies + Runtime → Container

Mengapa Containerization Dibutuhkan?

- Aplikasi backend membutuhkan runtime, dependency, database, konfigurasi, dan environment.
- Perbedaan environment dapat membuat aplikasi berjalan di satu komputer tetapi bermasalah di komputer lain.
- Containerization membantu menciptakan lingkungan aplikasi yang konsisten dan dapat direproduksi.

Apa Itu Docker?

- Platform untuk membangun, mendistribusikan, menjalankan, dan mengelola aplikasi menggunakan container.
- Membantu developer membuat environment yang konsisten dan terisolasi.
- Menjadi teknologi penting dalam modern backend development dan DevOps.

Container vs Virtual Machine

- Virtual Machine menjalankan sistem operasi lengkap di atas hypervisor.
- Container berbagi kernel sistem operasi host dan mengisolasi aplikasi serta environment-nya.
- Container umumnya lebih ringan, lebih cepat dijalankan, dan lebih efisien dalam penggunaan resource.

Konsep Inti: Image, Container, Dockerfile

Dockerfile

- Resep/instruksi untuk membangun image.

FROM → WORKDIR → COPY → RUN → EXPOSE → CMD

- Mendeskripsikan base image, direktori kerja, dependency, application files, port, dan proses startup.

• Membuat proses pembuatan environment aplikasi terdokumentasi dan reproducible.

Image

- Blueprint yang dapat digunakan berulang untuk membuat container.
- Berisi runtime, dependency, library, application files, dan konfigurasi dasar.
- Bersifat read-only, reusable, dan dapat didistribusikan
- Image belum berarti aplikasi sedang berjalan.

Container

- Instance dari image yang sedang/pernah dijalankan.
- Satu image dapat digunakan untuk membuat banyak container.
- Container dapat dibuat, dijalankan, dihentikan, dan dihapus
- Lifecycle: create → start → running → stop → remove.

Dockerfile → build → Image → run → Container

Container vs Virtual Machine

Virtual Machine

Menjalankan sistem operasi lengkap di atas hypervisor.

- Isolasi lebih berat
- Membawa guest OS
- Cocok ketika membutuhkan lingkungan OS yang terpisah

Container

Berbagi kernel host dan mengisolasi aplikasi + environment.

- Umumnya lebih ringan
- Start lebih cepat
- Cocok untuk packaging aplikasi dan environment

Docker Compose

- Digunakan untuk mendefinisikan dan menjalankan beberapa service/container sebagai satu aplikasi.
- Cocok untuk backend yang terdiri dari application service, database, cache, atau service lain.
- Menyederhanakan konfigurasi multi-container, network, volume, dan environment.

Tiga Konsep yang Akan Bertemu di Compose

Network

Menghubungkan service.

Dalam Compose, nama service dapat digunakan sebagai hostname.

Contoh: `DB_HOST=db`

Volume

Memisahkan data penting dari lifecycle container.

Database membutuhkan persistent storage agar data tidak hilang saat container dibuat ulang.

Environment

Konfigurasi di luar source code.

Contoh: DB_NAME, DB_USER, DB_PASSWORD, DB_HOST.

Demo Dosen — Verifikasi Docker


```
docker --version
docker compose version
docker info
docker run hello-world
docker ps
docker ps -a
docker images
```

Perhatikan

1. Apa yang terjadi setelah `docker run hello-world`?
2. Mengapa `docker ps` dan `docker ps -a` berbeda?
3. Mana image dan mana container?

Checkpoint kelas: mahasiswa dapat menjelaskan lifecycle dasar.

Guided Lab A — Container Lifecycle


```
docker run --name week3-nginx -d -p 8080:80 nginx
docker ps
# buka http://localhost:8080
docker stop week3-nginx
docker ps -a
docker start week3-nginx
docker logs week3-nginx
docker rm -f week3-nginx
```

Yang Diamati

- port mapping 8080:80
- state container
- logs
- stop/start
- remove

Inti: container bukan mesin permanen. Container dapat dibuat ulang.

Guided Lab B — Membuat Custom Python Image


```
week3-python-app/
├── app.py
├── requirements.txt
├── Dockerfile
└── .dockerignore
```

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
EXPOSE 8000
CMD ["python", "app.py"]
```

Aplikasi harus listen pada `0.0.0.0:8000` agar dapat diakses melalui network interface container.

Build → Run → Test


```
docker build -t week3-python-app:1.0 .
docker run --name week3-app -d -p 8000:8000
week3-python-app:1.0
docker logs week3-app
```

Uji

Buka:
http://localhost:8000/health

Periksa response JSON dan status aplikasi.

Checkpoint: jelaskan fungsi FROM, WORKDIR, COPY, EXPOSE, dan CMD

Guided Lab C — Environment Variable


```
docker rm -f week3-app
docker run --name week3-app -d \
  -p 8000:8000 \
  -e APP_ENV=container-lab \
  week3-python-app:1.0
```

Mengapa?

Konfigurasi dapat berbeda tanpa mengubah source code.

`.env` nyata tidak di-commit.
`.env.example` hanya berisi nama variable dan contoh non-rahasia.

Guided Lab D — Persistent Volume


```
docker volume create week3-data
docker volume ls
docker volume inspect week3-data
```

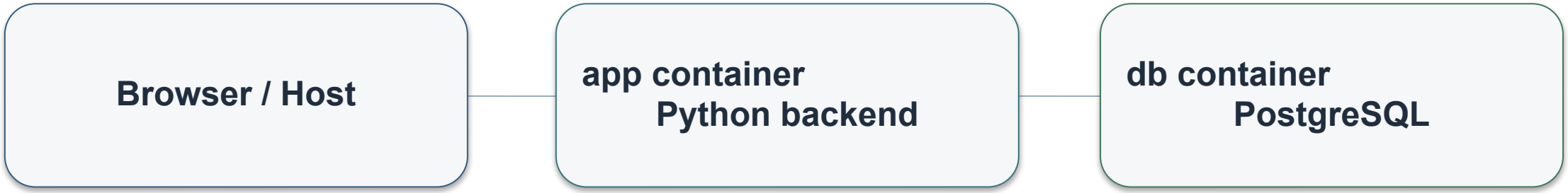
Inti Konsep

Volume menyimpan data di luar writable layer container.

Container boleh dihapus; data penting tetap dipertahankan melalui volume.

Database → Volume → Persistent Data

Guided Lab E — Docker Compose: App + PostgreSQL



PostgreSQL data → named volume

Compose: Satu File untuk Multi-Container


```yaml
services:
  app:
    build: .
    ports:
      - "8000:8000"
    env_file:
      - .env
    depends_on:
      - db

  db:
    image: postgres:17
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

Catatan Penting

`depends_on` mengatur urutan startup dasar, tetapi tidak menjamin database sudah siap menerima koneksi.

Untuk sistem matang: healthcheck + retry strategy.

Eksperimen — Membuktikan Persistent Data

- Jalankan stack.
- Buat tabel dan data uji di PostgreSQL.
- Jalankan `docker compose down`.
- Jalankan kembali `docker compose up -d`.
- Periksa data kembali.

```
CREATE TABLE demo (id SERIAL PRIMARY KEY, message
TEXT);
INSERT INTO demo(message) VALUES ('persistent
data');
SELECT * FROM demo;

# lalu
docker compose down
docker compose up -d
```

Jangan gunakan `docker compose down -v` jika ingin mempertahankan named volume.

Health Endpoint: Dari Backend Sederhana ke Stack


```
GET /health

{
  "app": "ok",
  "database": "ok"
}
```

Apa yang Dipelajari?

- aplikasi dapat mengecek dependency database
- response dapat memakai status 503 ketika database belum siap
- containerization memperkenalkan failure scenario nyata
- ini menjadi fondasi untuk health/readiness check pada tahap deployment berikutnya

Dockerfile & Environment: Prinsip Dasar

- Gunakan base image dan tag.
- Susun layer agar build cache dapat dimanfaatkan.
- Gunakan `.dockerignore`.
- Jangan memasukkan `.env`, secret, `.git`, `.venv`, atau file tidak relevan ke build context.
- Jangan menyimpan data persisten pada writable layer container.
- Pisahkan development configuration dan production configuration.
- Untuk production, pertimbangkan non-root user.

Multi-Stage Build — Cukup Dipahami Konsepnya


```
FROM ... AS builder
# build / install artifacts

FROM ... AS runtime
# copy only required runtime artifacts
# run application
```

Untuk Minggu 3

Pada Python sederhana, manfaatnya belum tentu besar.

Yang penting dipahami:
- ada stage build
- ada stage runtime
- image akhir dapat membawa hanya artefak yang diperlukan

.

Hubungan Docker dengan project Simple LMS

Minggu 2

Simple LMS dirancang sebagai modular monolith:

Users
Courses
Enrollments
Lessons
Assignments
Submissions

Minggu 3

Environment dikemas:

Python app
+ Docker image
+ container
+ network
+ volume
+ Compose
+ PostgreSQL latihan

Minggu 4

Mulai implementasi backend sebenarnya dengan:

Django
Django ORM
Models
Relationships
Database

Alur Praktikum Hari Ini

Minggu 3 | Docker Containerization

01
Konsep Docker

02
Demo hello-world

03
Lifecycle Nginx

04
Dockerfile Python

05
Build & Run

06
Environment Variable

07
Volume

08
Compose + PostgreSQL

09
Network

10
Persistent Data

11
Konsep Multi-stage

12
Diskusi

Ringkasan

Docker = fondasi environment yang konsisten dan dapat direproduksi.

- Dockerfile membangun image.
- Image menjadi dasar container.
- Network menghubungkan service.
- Volume menjaga data persisten.
- Environment variable memisahkan konfigurasi.
- Compose mengelola multi-container application.
- Praktik minggu ini belum mengubah domain Simple LMS; kita menyiapkan environment-nya.